PERSONAL DEMO PROJECT · SAMPLE APP AND DATA

Fix report: Huddle Notes

Security and sign-in review
Next.js + Supabase · October 2026

Summary. Before this work, anyone on the internet could read, change or delete every user's notes without logging in, and the app handed every visitor a key that controls the whole database. Both problems are fixed, along with sign-up and error handling. Privacy is now enforced by the database itself, and 15 automated tests check it on every change.

Two things still need you: replace the leaked key and set your site address in Supabase (see "What you need to do").

What was wrong, and where it stands

RISK	WHAT IT MEANT FOR YOUR USERS	WHAT WAS DONE	STATUS
Critical	Anyone could read, edit or delete any user's notes. No account needed.	The database now checks every request and returns only the signed-in user's own notes.	**Fixed**
Critical	The master database key was included in the website's code, where any visitor could copy it.	The key now stays on the server and is used for one task only (deleting your own account).	**Fixed***
High	Changing the number in a note's web address opened someone else's note, even when logged out.	Other people's notes now show "Note not found". Logged-out visitors are sent to the login page.	**Fixed**
Medium	A user could save a note into another person's account.	The database, not the browser, now decides who owns each note.	**Fixed**
Medium	New users were dropped on the login page with no explanation, and the confirmation email did not bring them into the app.	Sign-up now says "Check your email", and the link signs the user in and opens their notes.	**Fixed***
Low	Errors showed nothing: a wrong password looked the same as a broken site.	Clear error messages, plus loading and "no notes yet" screens.	**Fixed**

* Complete in the code. Needs the matching step under "What you need to do" on the live project.

How you know it works

- The same 15 tests were run before and after. **Before: 4 passed, 11 failed. After: 15 passed.**
- Five of them act like an attacker: they skip the website and ask the database directly, using a second user's login or no login at all. They pass only when the database refuses.
- The tests run automatically on every future code change, so this cannot quietly break again.
- Everything that worked before still works: logging in, writing, editing and deleting your own notes, deleting your account.

What you need to do

1. **Replace the leaked key.** In Supabase, generate a new service-role key and save it as `SUPABASE_SERVICE_ROLE_KEY` on your host. The old one was public, so it must be treated as stolen.
2. **Set your site address** in Supabase under Authentication → URL Configuration, so the links in confirmation emails point to your site instead of a developer's computer.

What changed for your data

No notes were deleted. Any note that had no valid owner was moved to a locked holding table (`notes_orphaned`) for you to review, because it could not safely be shown to anyone.

Not covered

Password reset (the app does not have one yet), limits on repeated login attempts, two-step login, and a review of your hosting settings. None are urgent; password reset is the one users will ask for first.

Full technical detail: docs/AUDIT.md (each issue with steps to reproduce) and the pull request "Fix auth and Row Level Security". This report describes a demo project built to show the process; it is not a client engagement.